

ECommerce Sales Analytics on ClickHouse

DSCI 551 Course Project — Phase 2: Midterm Report

Team Members: Wenxin Li, Isabelle Du, Xinyuan Huang

Date: March 20, 2026

1. Database System Overview

ClickHouse is an open-source columnar database built for analytical queries. It was originally developed by Yandex around 2009 for its Metrika web analytics platform and was released as open source in 2016. The engine design is described in detail by Schulze et al. (2024). At a high level, ClickHouse combines columnar storage, vectorized execution, and distributed processing to support fast aggregation over large datasets.

Compared with MySQL and MongoDB, two architectural differences are especially important for this project. First, each column of a table sits in its own file on disk. A query computing `sum(price * quantity)` reads only the price and quantity files; it never loads category, city, or payment_method. In contrast, MySQL's InnoDB stores all columns of a row together in the same 16 KB page, so the engine may read data that the query does not actually need. Second, ClickHouse processes data in batches of up to 8192 values per column rather than looping through rows one at a time. This batch-oriented pipeline keeps the CPU busy on tight loops and avoids the per-row overhead that row-oriented engines pay. This design is especially useful for e-commerce analytics, where queries often aggregate a small set of numeric columns across millions of rows.

2. Focus Area A — Storage and Compression

ClickHouse writes data through its MergeTree engine. Each INSERT creates an immutable, sorted part on disk, and background merge threads combine smaller parts into larger ones over time. This behavior resembles an LSM-style design, although ClickHouse does not organize parts into fixed levels (Schulze et al., 2024). The ORDER BY clause in the DDL controls how rows inside each part are sorted.

Unlike row-oriented systems, which store all fields of a row together in the same page, ClickHouse stores and compresses each column separately. This design is important for analytical workloads because queries often read only a few relevant columns instead of the

full row. In our project, the table `orders_local` declares `PARTITION BY toYYYYMM(order_date)`, splitting data into one partition per calendar month. Parts belonging to different partitions never get merged together. After loading one million rows, shard 1 holds 24 partitions (January 2023 through December 2024). Each partition contains roughly 20000 rows.

Any query that filters on `order_date` can skip entire partitions at the metadata level, before the engine even opens a column file. This reduces unnecessary disk reads and improves query efficiency for time-based analysis.

Compression varies column by column depending on data type and value distribution. `region` holds just six distinct strings, and because the rows are sorted by `region` first (it's the leading `ORDER BY` column), long runs of identical values appear on disk. LZ4 squeezes those runs at a 23.4× ratio. `price`, on the other hand, stores near-random floats and compresses at under 2×. The application queries `system.parts_columns` and prints these numbers so the link between data characteristics and disk footprint is visible.

3. Focus Area B — Indexing and Data Skipping

MergeTree does not maintain a traditional per-row B-tree index. Instead, it stores one primary-key sample per granule, with 8,192 rows per granule by default, creating a sparse index (ClickHouse Documentation, n.d.). At query time, the engine binary-searches these samples to identify which granules may contain matching rows and reads only those granules. Every other granule gets skipped without any disk access.

This design differs from row-oriented systems that often rely on B-tree indexes over individual row keys. In ClickHouse, pruning happens at the granule level rather than the row level, which is more efficient for large analytical scans. In our project, the table `orders_local` uses `ORDER BY (region, order_date, user_id)`. A filter like `region = 'Asia' AND order_date BETWEEN '2023-06-01' AND '2023-06-30'` lines up with the first two key columns, and the engine narrows the scan to a handful of granules. A filter on `city = 'Tokyo'` cannot use the sparse index at all because `city` is not in the `ORDER BY`, so every granule must be checked. `EXPLAIN` output confirms this difference in pruning effectiveness. The primary-key-aligned query reads 1 out of 74 granules, while the city

query reads all 74. Two skip indexes add another pruning layer. A bloom filter index on `user_id` (granularity 4) lets the engine test whether a given user ID could exist in a granule group without decompressing the column. A minmax index on `price` (granularity 4) records the min and max price per group, so a range filter like `price >= 400 AND price <= 500` can drop any group whose max price sits below 400.

Our application compares how many granules are read when a query matches the `ORDER BY` prefix versus when it does not, and it prints the corresponding `EXPLAIN` results so that the effect of sparse indexing and data skipping is visible.

4. Focus Area C — Distribution

Our deployment uses two ClickHouse containers as separate shards on ports 8123 and 8124, coordinated by a ZooKeeper instance. Each shard stores data in a local `ReplicatedMergeTree` table named `orders_local`, while a `Distributed` table named `orders` sits on top and routes queries across both shards.

Inserts are issued through the `Distributed` table. ClickHouse hashes each row's `user_id` with `xxHash64` to determine which shard should store it. After loading one million rows, shard 1 contains about 501,000 rows and shard 2 contains about 499,000 rows. This indicates near-even shard balancing.

Reads are distributed across both shards in parallel. Each shard executes the query locally and returns partial results to the coordinator, which merges them into the final output. Unlike a single-node database system, this design allows ClickHouse to parallelize scans and partial aggregation across machines before combining the results. `EXPLAIN PIPELINE` output makes this execution pattern visible. For a `GROUP BY` query, the pipeline shows a `ReadFromRemote` operator feeding into `MergingAggregated`, which reflects ClickHouse's two-phase aggregation strategy: each shard performs local pre-aggregation first, and the coordinator then merges the aggregate states into the final result.

5. Preliminary Application Design

The application is a Python command-line interface (CLI) with sixteen numbered features. A user selects a feature from the menu, optionally enters parameters such as a date range,

user ID, or price bounds, and then views both the query result and the corresponding EXPLAIN output in the same interface. The design goal is to present the SQL query alongside the engine’s internal access path so that application behavior can be directly connected to database internals.

Working features as of this checkpoint include regional sales aggregation (columnar scan), time-range filtering (partition pruning), user lookup (bloom filter), batch insert (MergeTree write path), partition stats (system.parts), column compression (system.parts_columns), cluster info (system.clusters), shard distribution (per-shard row counts), and index-vs.-full-scan comparison (EXPLAIN granule counts). The remaining features—monthly trend, category stats, HyperLogLog, top users, price-range filter, cross-shard aggregation, and query log stats—are coded and have passed manual testing.

6. Preliminary Mapping: DB Internals ↔ Application

A subset of the mappings between features and ClickHouse internals is listed below. In the final report, we will expand each row with EXPLAIN output and a short explanation of why the internal mechanism matters for the corresponding query.

Table 1 Preliminary mapping between application features and ClickHouse internals

#	Application Feature	ClickHouse Internal
1	Regional sales aggregation	Columnar scan, vectorized pipeline
2	Time-range query	Partition pruning, sparse index
3	User order lookup	Bloom filter skip index
8	Price range filter	MinMax skip index
10	Batch insert	MergeTree immutable-part write
12	Column compression stats	Per-column compression ratio
14	Shard distribution	Distributed table hash routing
16	Index vs full scan	Primary key granule skipping vs full read

7. Progress and Challenges

The Docker cluster, schema, data generator, and all sixteen CLI features have been implemented and run successfully in our testing. One problem showed up during development: `system.parts_columns` returned zero for both compressed and uncompressed byte counts. ClickHouse was storing the small parts in “compact” format, which bundles all columns into one file and does not expose per-column size counters. Setting `min_bytes_for_wide_part = 0` in the table DDL forced “wide” format (one file per column), and the byte counts became valid. In production this setting would not be ideal — compact format reduces the number of file descriptors when a table has many small parts — but for a project that needs to show compression ratios, wide format is the right call.

Remaining work includes expanding the final report sections for each mapping and generating the plots that will appear in the report figures. Based on the Phase 1 feedback, we are also prepared to use a smaller demo-sized dataset if needed to keep the live demonstration responsive.

References

Schulze, R., Schreiber, T., Yatsishin, I., Dahimene, R., & Milovidov, A. (2024).

ClickHouse—Lightning fast analytics for everyone. Proceedings of the VLDB Endowment, 17(12), 3731–3744.

ClickHouse Documentation. (n.d.). *MergeTree engine family*. ClickHouse.

<https://clickhouse.com/docs/en/engines/table-engines/mergetree-family/mergetree>